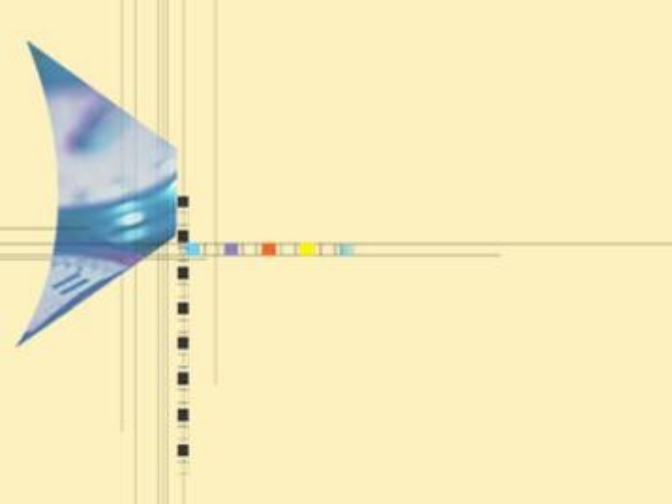




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025第二学期，1-16周

周一(3-4，三区1-502)，周三（6-7，三区1-325）



主要内容

4.1 引言

4.2 动态规划算法的简单例

4.3 动态规划算法设计框架

4.4 动态规划策略的典型应用



主要内容

4.4 动态规划策略的典型应用（选讲）

4.4.1 所有点对最短路径问题

4.4.2 0/1背包问题

4.4.3 图像压缩问题

4.4.4 最大子段和

4.4.5 多段图问题

4.4.6 最优二分检索树问题

4.4.7 投资问题

4.4.1 所有点对最短路径问题

问题描述:

- 设 $G = (V, E)$ 是一个有向图，其中的每条边 (i, j) 有一个非负的长度 $l[i, j]$ ，如果从顶点 i 到顶点 j 没有边，则 $l[i, j] = \infty$ 。
- 要求找出从每个顶点到其他所有顶点的距离，这里，从顶点 x 到顶点 y 的距离是指从 x 到 y 的最短路径的长度。
- 如果定义一个矩阵 $D = \{d_{i,j}\}$ ，其中 $d_{i,j} > 0$ 表示 i 城市到 j 城市的距离。 $d_{i,i} = 0$ ，表示任意城市没有到自己的回路。若 i 与 j 之间无路可通，那么 $d_{i,j}$ 就是无穷大。那么问题就是要求求这个距离矩阵 D ，找出任意两个城市之间的最短路径。



4.4.1 所有点对最短路径问题

- 如何找出最短路径呢？这里先将问题分解：
 - 对于任何一个城市 i 而言， i 到 j 的最短路径不外乎存在经过 i 与 j 之间的 k 和不经 k 两种可能。
 - 所以可以令 $k=1, 2, 3, \dots, n$ (n 是城市的数目)，再检查 $d_{i,j}$ 与 $d_{i,k} + d_{k,j}$ 的值； $d_{i,k}$ 与 $d_{k,j}$ 分别是目前为止所知道的 i 到 k 与 k 到 j 的最短距离，因此 $d_{i,k} + d_{k,j}$ 就是 i 到 j 经过 k 的最短距离。每当一个 k 查完了， $d_{i,j}$ 就是目前的 i 到 j 的最短距离。
 - 重复这一过程，最后当递归查完所有的 k 时， $d_{i,j}$ 里面存放的就是 i 到 j 之间的最短距离了。

定义求最优值的递推式

- 设 i 和 j 是 V 中两个不同的顶点,定义 $d_{i,j}^k$ 是从 i 到 j ,并且不经过 $\{k+1, k+2, \dots, n\}$ 中任何顶点的最短路径的长度。
- $d_{i,j}^0 = l[i, j]$, $d_{i,j}^1$ 是从 i 到 j ,除了可能经过顶点1以外,不经过任何其他顶点的最短路径。
- $d_{i,j}^2$ 是从 i 到 j ,除了可能经过顶点1、顶点2或同时经过它们以外,不经过任何其他顶点的最短路径,等等。
- 这样由定义可知, $d_{i,j}^n$ 是从 i 到 j 的最短路径长度, $d_{i,j}^n$ 也就是从 i 到 j 的距离。给出这个定义,可以递归地计算

$$d_{i,j}^k = \begin{cases} l[i, j] & \text{if } k = 0 \\ \min\{d_{i,j}^{k-1}, d_{i,k}^{k-1} + d_{k,j}^{k-1}\} & \text{if } 1 \leq k \leq n \end{cases}$$

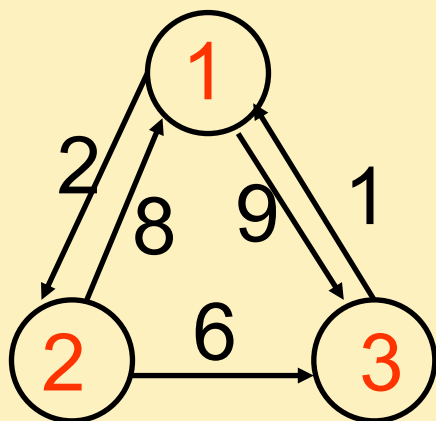
Floyd算法

- 下面是Floyd设计出的算法，用自底向上地解上面的递推式的方法来处理。
- 它用 $n+1$ 个 $n \times n$ 维矩阵 $D_0, D_1, \dots, D_n$ 来计算最短约束路径的长度。
- 开始时，置 $D_0[i,i] = 0$ ；如果 $i \neq j$ 并且 (i,j) 是 G 中的边，则 $D_0[i,j] = l[i,j]$ ；否则置 $D_0[i,j] = \infty$ 。
- 然后执行 n 次迭代，使在第 k 次迭代后， $D_k[i,j]$ 含有从顶点 i 到顶点 j ，且不经过编号大于 k 的任何顶点的最短路径的长度。
- 这样在第 k 次迭代中，可以用公式计算 $D_k[i,j]$ ：

$$D_k[i,j] = \min \{ D_{k-1}[i,j], D_{k-1}[i,k] + D_{k-1}[k,j] \}$$

例子

考虑下图所示的有向图



矩阵 D_0, D_1, D_2 和 D_3 分别是:

$$D_0 = \begin{bmatrix} 0 & 2 & 9 \\ 8 & 0 & 6 \\ 1 & \infty & 0 \end{bmatrix}$$

$$D_1 = \begin{bmatrix} 0 & 2 & 9 \\ 8 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

$$D_2 = \begin{bmatrix} 0 & 2 & 8 \\ 8 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

$$D_3 = \begin{bmatrix} 0 & 2 & 8 \\ 7 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$



Floyd算法

- 一个重要的发现是，在第 k 次迭代中第 k 行和第 k 列都是不变的；
- 因此，可以直接在 D 矩阵完成更新就可以用，即仅用一个 D 矩阵进行计算，不需要使用另外矩阵来保存上次迭代的计算结果值。
- 最终，仅用一个 $n \times n$ 矩阵执行这个计算的算法由算法 FLOYD 给出。

算法4.7 FLOYD

输入: $n \times n$ 维矩阵 $l[1..n, 1..n]$, 对于有向图 $G = (\{1, 2, \dots, n\}, E)$ 中的边 (i, j) 的长度是 $l[i, j]$ 。

输出: 矩阵 D 使得 $D[i, j] = i$ 到 j 的距离。

```
1.  $D \leftarrow l$       {将输入矩阵  $l$  复制到  $D$ }
2. for  $k \leftarrow 1$  to  $n$ 
3.   for  $i \leftarrow 1$  to  $n$ 
4.     for  $j \leftarrow 1$  to  $n$ 
5.        $D[i, j] = \min\{D[i, j], D[i, k] + D[k, j]\}$ 
6.     end for
7.   end for
8. end for
```

- 显然, 算法的运行时间是 $\Theta(n^3)$, 它的空间复杂性是 $\Theta(n^2)$ 。

4.4.2 0/1背包问题

- 背包问题可以定义如下:
 - 设 $U = \{u_1, u_2, \dots, u_n\}$ 是一个准备放入容量为 C 的背包中的 n 项物品的集合。
 - 对于 $1 \leq j \leq n$, 令 s_j 和 v_j 分别为第 j 项物品的体积和价值, 这里, C, s_j, v_j 和 $1 \leq j \leq n$ 都是正整数
 - 我们要解决的问题是选择 U 中的一些物品来装背包, 这些物品的总体积不超过 C , 然而要使它们的总价值最大。
- 更形式地, 给出有 n 项物品的 U , 我们要找出一个子集合 S , 使得 $\sum_{u_i \in S} v_i$ 在约束条件 $\sum_{u_i \in S} s_i \leq C$ 下最大。



4.3.2 0/1背包问题—基本思想

- 设 $V[i,j]$ 用来表示从前 i 项 $\{u_1, \dots, u_i\}$ 中取出来的装入体积为 j 的背包的物品的 **最大价值**。这里， i 的范围是从 0 到 n ， j 的范围是 0 到 C 。
- $V[0,j]$ 对于所有 j 的值是 0，这是由于背包中什么也没有。另一方面， $V[i,0]$ 对于所有 i 的值为 0，因为没有东西可放到为 0 的背包里。

4.4.2 0/1背包问题—基本思想

- **0/1背包问题**特点是：每种物品仅有一件，可以选择放或不放。
- **用子问题定义状态**：即 $V[i,j]$ 表示前 i 件物品放入一个容量为 j 的背包可以获得的最大价值。
- 针对“将前 i 件物品放入容量为 j 的背包中”这个子问题，若只考虑第 i 件物品的策略（放或不放），那么就可以转化为一个只牵扯前 $i-1$ 件物品的问题。
 - 如果不放第 i 件物品，那么问题就转化为“前 $i-1$ 件物品放入容量为 j 的背包中”，价值为 $V[i-1,j]$ ；
 - 如果放第 i 件物品，那么问题就转化为“前 $i-1$ 件物品放入剩下的容量为 $j-s_i$ 的背包中”，此时能获得的最大价值就是 $V[i-1, j-s_i]$ 再加上通过放入第 i 件物品获得的价值 v_i 。

4.3.2 0/1背包问题—观察结论

$V[i,j]$ 是下面两个量的最大值:

1. $V[i-1,j]$:仅用最优的方法取自 $\{u_1, u_2, \dots, u_{i-1}\}$ 的物品去装入体积为 j 的背包所得到的价值最大值。
2. $V[i-1,j-s_i]+v_i$:用最优的方法取自 $\{u_1, u_2, \dots, u_{i-1}\}$ 的物品去装入体积为 $j-s_i$ 的背包所得到的价值最大值加上物品 u_i 的价值。这仅应用于如果 $j \geq s_i$ 以及把物品 u_i 加到背包上的情况。

$$V[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ V[i-1, j] & \text{if } j < s_i \\ \max \{V[i-1, j], V[i-1, j-s_i] + v_i\} & \text{if } i > 0 \text{ and } j \geq s_i \end{cases}$$

算法4.8 KNAPSACK

输入: 物品集合 $U=\{u_1, u_2, \dots, u_n\}$, 体积分别为 $s_1, s_2, \dots, s_n$, 价值分别为 $v_1, v_2, \dots, v_n$, 容量为 C 的背包。

输出: $\sum_{u_i \in S} v_i$ 的最大总价值, 且满足 $\sum_{u_i \in S} s_i \leq C$, 其中 $S \subseteq U$.

```
1. for i ← 0 to n
2.      $V[i, 0] \leftarrow 0$ 
3. end for
4. for j ← 0 to n
5.      $V[0, j] \leftarrow 0$ 
6. end for
7. for i ← 1 to n
8.     for j ← 1 to C
9.          $V[i, j] \leftarrow V[i-1, j]$ 
10.        if  $s_i \leq j$  then  $V[i, j] \leftarrow \max\{V[i, j], V[i-1, j-s_i] + v_i\}$ 
11.    end for
12. end for
13. return  $V[n, C]$ 
```



例 容量: 9; 4 种体积的物品: 2, 3, 4, 5; 价值分别为: 3, 4, 5, 7.

The maximum value is 12 and there are two optimal solutions:
1,2,3 and 3,4.

	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7	7	7
3	0	0	3	4	5	7	8	9	9	12
4	0	0	3	4	5	7	8	10	11	12



算法分析

- 很明显, 由于计算表的每一项需要 $\Theta(1)$ 时间, 算法的时间复杂度恰好是表的大小 $\Theta(nC)$ 。
- 由于计算当前行只需要上一次计算的行, 因此对算法稍作修改可以使其只需要 $\Theta(C)$ 的空间。
- 定理4.3
 - 背包问题的最优解能够在 $\Theta(nC)$ 的时间和 $\Theta(C)$ 的空间内得到。

4.4.3 图像压缩问题

计算机中的图像：一系列像点构成，每个像点称为一个像素。每个像点用一个整数表示其颜色值。

例，对黑白图像，每个像点具有一个灰度值，用0~255之间整数表示，如果采用相同二进制位数来表示，每个整数需要8位二进制位。假设图像有n个像点。则有：

像素点灰度值：0~255，表示为8位二进制数

像素点灰度值序列： $\langle p_1, p_2, \dots, p_n \rangle$ ， p_i 为第*i*个像素点灰度值

存储空间：一共需要 $8n$ 个二进制位。--无压缩图像

一般而言，一幅图像中许多连续区域中像点灰度值接近，如交通标志图片，大片区域是白色，少数区域有比较单调的颜色。对这样的图像可以采用分段存储的方法：对灰度值较小的段的像素采用比较少的位数（比如2位），对灰度值较大的段的像素采用较多的位数（比如8位），减少图像存储空间的占用。

----变位压缩技术



变位压缩技术

无压缩存储图像的读取：由于每个像素8位，因此，每读8位就是一个像素的灰度值。

采用变位压缩技术存储的图像读取：当读取一个0/1序列时，如何知道每段的划分位置以及每一段像素占用的二进制位数？因此需要对段的划分和段中像素适用的二进制位数（同一个段内像素用的存储位数一样）给出明确信息。

解决：

每一段给出两个整数值，一个表示该段含有的像素个数，一个表示每个像素所占用的二进制位数。比如第 i 段，有 $L[i]$ 个像素，每个像素用 $b[i]$ 位。

规定 $L[i] \leq 256$ ，即像素个数不超过256个，因此，可用8位二进制位表示 $L[i]$ ；因每个灰度值在 $0 \sim 255$ 之间，因此每个灰度值所用二进制的位数 $b[i]$ 不超过8，因此可用3位表示 $b[i]$ ；

因此，对每一段，增加额外的11个二进制位作为段头。

图像压缩

像素点灰度值: 0~255, 表示为 8 位二进制数

像素点灰度值序列: $\langle p_1, p_2, \dots, p_n \rangle$, p_i 为第 i 个像素点灰度值

变位压缩存储格式: 将 $\langle p_1, p_2, \dots, p_n \rangle$ 分割 m 段 $S_1, S_2, \dots, S_m$
 i 段有 $L[i]$ 个像素, 每个像素 $b[i]$ 位, h_i 为该段最大像素的位数

$$h_i \leq b[i] \leq 8, \quad h_i = \left\lceil \log(\max_{p_k \in S_i} p_k + 1) \right\rceil$$

约束条件: 每段像素个数 $L[i] \leq 256$

段头11位: $b[i]$ 的二进制表示(3 位) + $L[i]$ 的二进制表示(8位)

i 段占用空间: $b[i] \times L[i] + 11$

问题: 给定像素序列 $\langle p_1, p_2, \dots, p_n \rangle$, 确定最优分段, 即

$$\min_T \left\{ \sum_{i=1}^j (b[i] * L[i] + 11) \right\}, T = \{S_1, S_2, \dots, S_j\} \text{ 为分段}$$

实例

灰度值序列 $P=<10,12,15,255,1,2,1,1,2,2,1,1>$

分法1: $S_1=<10,12,15>$, $S_2=<255>$, $S_3=<1,2,1,1,2,2,1,1>$

分法2: $S_1=<10,12,15,255,1,2,1,1,2,2,1,1>$

分法3: 分成12组, 每组一个数

存储空间

分法1: $11 \times 3 + 4 \times 3 + 8 \times 1 + 2 \times 8 = 69$

分法2: $11 \times 1 + 8 \times 12 = 107$

分法3: $11 \times 12 + 4 \times 3 + 8 \times 1 + 1 \times 5 + 2 \times 3 = 163$

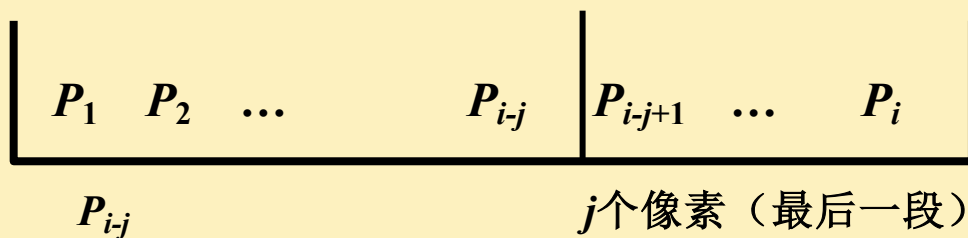
结论: 分法1是其中最优的分法

算法设计

考虑子问题的划分边界

每个子问题的输入都是从像素值 p_1 开始。第 i 个子问题 P_i 是到第 i 个像素值结束。即有 $P_i = \langle p_1, p_2, \dots, p_i \rangle$

如果最后一段的灰度值有 j 个，其中 $1 \leq j \leq \min\{256, i\}$ ，那么剩余部分将归约为输入是 $P_{i-j} = \langle p_1, p_2, \dots, p_{i-j} \rangle$ 的子问题。



算法设计

递推方程

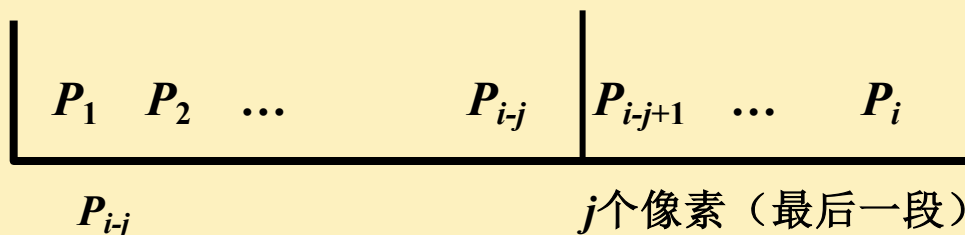
设 $s[i]$ 是像素序列有 $P_i = \langle p_1, p_2, \dots, p_i \rangle$ 的最优分段所需存储总位数，设最后一段是第 m 段，记作 S_m ，且 S_m 含有 j 个灰度值，存储 S_m 的每个灰度值需要的二进制位数记做 $b[i-j+1, i]$

$$s[0] = 0; s[1] = b[1, 1] + 11$$

$$s[i] = \min_{1 \leq j \leq \min\{i, 256\}} \{s[i-j] + j \times b[i-j+1, i] + 11\}$$

$$b[i-j+1, i] = \left\lceil \log(\max_{p_k \in S_m} p_k + 1) \right\rceil \leq 8$$

最后一段 S_m 中每个像素点占用位数为其中最大灰度值占用的位数



算法

算法4.9 Compress (P, n) //计算含有 n 个像素点图像最优分段的最小位数 $S[n]$

1. $Lmax \leftarrow 256$; $header \leftarrow 11$; $S[0] \leftarrow 0$ //最大段长 $Lmax$, 头 $header$
2. for $i \leftarrow 1$ to n do
3. $b[i] \leftarrow \text{length}(P[i])$ // $b[i]$ 是第 i 个像素灰度 $P[i]$ 的二进制位数
4. $bmax \leftarrow b[i]$ //3-6行分法的最后一段只有 $P[i]$ 自己, 最大位数
5. $S[i] \leftarrow S[i-1] + bmax$
6. $l[i] \leftarrow 1$ // $l[i]$ 是达到最少位数时最后一段的灰度值个数, 初始为1
7. for $j \leftarrow 2$ to $\min\{i, Lmax\}$ do //最后段含 j 个像素
8. if $bmax < b[i-j+1]$ //统一段内表示像素的二进制位数, 最后一段最大位数
9. then $bmax \leftarrow b[i-j+1]$
10. end if
11. if $S[i] > S[i-j] + j * bmax$ //找到更少的位数
12. then $S[i] \leftarrow S[i-j] + j * bmax$
13. $l[i] \leftarrow j$
14. end if
15. end for
16. $S[i] \leftarrow S[i] + header$ // $P[1], \dots, P[i]$ 最优分段的最小位数
17. end for

时间复杂度 $T(n) = O(n)$

$P = \langle 10, 12, 15, 255, 1, 2 \rangle$.

已经计算: $S[1]=15, S[2]=19, S[3]=23, S[4]=42, S[5]=50$

$l[1]=1, l[2]=2, l[3]=3, l[4]=1, l[5]=2$

$S[6]=63$

10	12	15	255	1	2
----	----	----	-----	---	---

$S[5]=50$

$1 \times 2 + 11$

$S[6]=57$

10	12	15	255	1	2
----	----	----	-----	---	---

$S[4]=42$

$2 \times 2 + 11$

$S[6]=58$

10	12	15	255	1	2
----	----	----	-----	---	---

$S[3]=23$

$3 \times 8 + 11$

$S[6]=62$

10	12	15	255	1	2
----	----	----	-----	---	---

$S[2]=19$

$4 \times 8 + 11$

$S[6]=66$

10	12	15	255	1	2
----	----	----	-----	---	---

$S[1]=15$

$5 \times 8 + 11$

$S[6]=59$

10	12	15	255	1	2
----	----	----	-----	---	---

$6 \times 8 + 11$

构造解

算法4.10 Traceback(n, l)

输入：数组 l // $l[j]$ 表示 $P[1], \dots, P[i]$ 最优分段最后一段的像素个数

输出：数组 C // $C[j]$ 是从后向前追踪的第 j 段的长度

1. $j \leftarrow 1$ // j 为正在追踪的段数
2. while $n \neq 0$ do
3. $C[j] \leftarrow l[n]$
4. $n \leftarrow n - l[n]$
5. $j \leftarrow j + 1$
6. end while

时间复杂度： $O(n)$

上例中： $C[1] = l[6] = 2$, $C[2] = l[6 - l[6]] = l[4] = 1$; $C[3] = l[4 - l[4]] = l[3] = 3$
最优分段：{10,12,15},{255},{1,2}



4.4.4 最大子段和

问题：给定 n 个整数（可以为负数）的序列
 $(a_1, a_2, \dots, a_n)$

求 $\max\{0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k\}$

实例： $(-2, 11, -4, 13, -5, -2)$

解：最大子段和 $a_2 + a_3 + a_4 = 20$

算法1---顺序求和+比较

算法2---分治策略

算法3---动态规划

算法1 顺序求和+比较

算法4.11 Enumerate

输入：数组 $A[1..n]$

输出： $sum, first, last$

1. $sum \leftarrow 0$
2. for $i \leftarrow 1$ to n do // i 为当前和的首位置
3. for $j \leftarrow i$ to n do // j 为当前和的末位置
4. $thissum \leftarrow 0$ // $thissum$ 为 $A[i]$ 到 $A[j]$ 之和
5. for $k \leftarrow i$ to j do
6. $thissum \leftarrow thissum + A[k]$
7. if $thissum > sum$
8. then $sum \leftarrow thissum$
9. $first \leftarrow i$ // 记录最大和的首位置
10. $last \leftarrow j$ // 记录最大和的末位置

时间复杂度： $O(n^3)$

算法2 分治策略

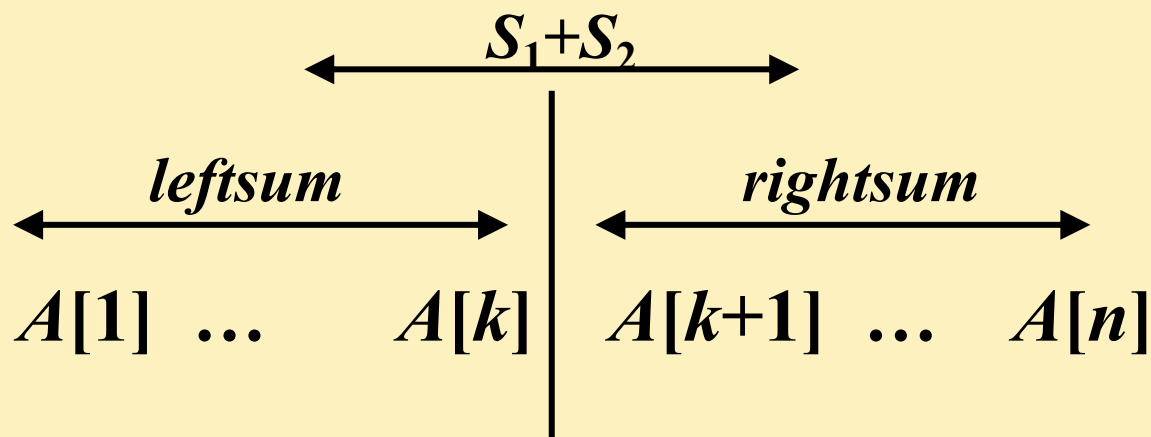
将序列分成左右两半，中间分点 $center$

递归计算左段最大子段和 $leftsum$

递归计算右段最大子段和 $rightsum$

$a_{center} \rightarrow a_1$ 的最大和 S_1 , $a_{center+1} \rightarrow a_n$ 的最大和 S_2

$\max \{ leftsum, rightsum, S_1+S_2 \}$



分治算法

算法4.12 MaxSubSum($A, left, right$)

输入：数组 A , $left$, $right$ 分别是 A 的左、右边界

输出： A 的最大子段和 sum 及其子段的前后边界

1. if $|A|=1$ then 输出元素值（当值为负时输出0）
2. $center \leftarrow \lfloor (left+right)/2 \rfloor$
3. $leftsum \leftarrow \text{MaxSubSum}(A, left, center)$ //子问题 A_1
4. $rightsum \leftarrow \text{MaxSubSum}(A, center+1, right)$ //子问题 A_2
5. $S_1 \leftarrow$ 从 $A[center]$ 向左的最大和 //从 $center$ 向左的最大和
6. $S_2 \leftarrow$ 从 $A[center+1]$ 向右的最大和 //从 $center+1$ 向右的最大和
7. $sum \leftarrow S_1 + S_2$
8. if $leftsum > sum$ then $sum \leftarrow leftsum$
9. if $rightsum > sum$ then $sum \leftarrow rightsum$

时间： $T(n)=2T(n/2)+O(n)$, $T(c)=O(1)$

$T(n)=O(n\log n)$

算法3： 动态规划

令 $C[i]$ 是 $A[1..i]$ 中必须包含元素 $A[i]$ 的最大子段和

$$C[i] = \max_{1 \leq k \leq i} \left\{ \sum_{j=k}^i A[j] \right\}$$



递推方程: $C[i] = \max\{C[i-1] + A[i], A[i]\} \quad i=1, \dots, n$

$C[1] = A[1]$ 若 $A[1] > 0$, 否则 $C[1] = 0$

解: $\text{OPT}(A) = \max_{1 \leq i \leq n} \{C[i]\}$

算法 MaxSum

算法4.13 MaxSum(A, n)

输入：数组 A

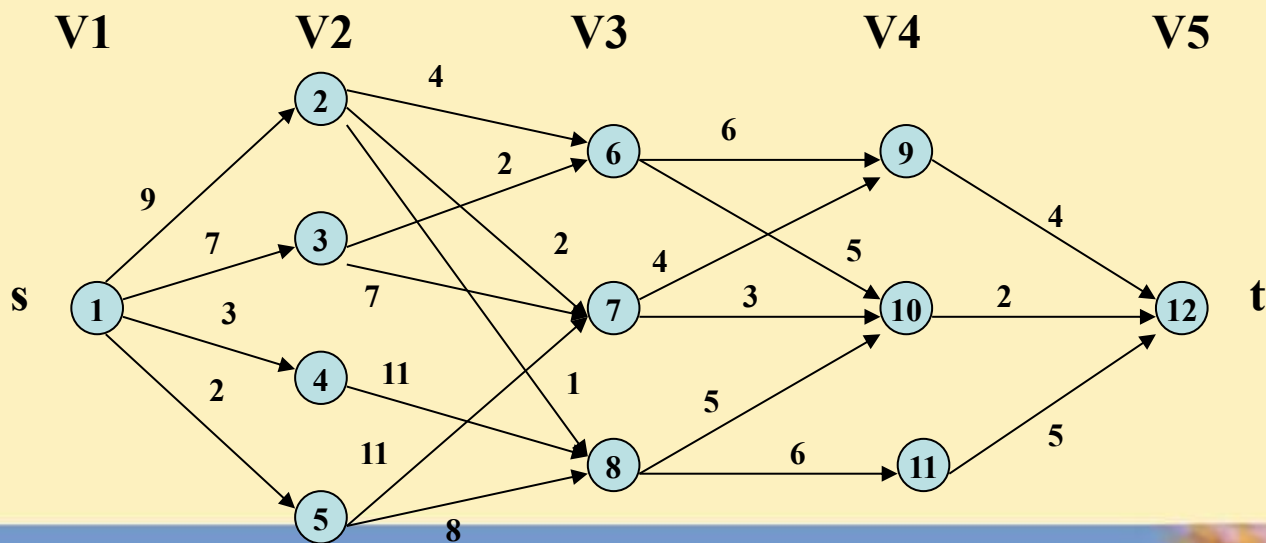
输出：最大子段和 sum , 子段的最后位置 c

1. $sum \leftarrow 0$
2. $b \leftarrow 0$ // b 是前一个最大子段和
3. for $i \leftarrow 1$ to n do
4. if $b > 0$
5. then $b \leftarrow b + A[i]$
6. else $b \leftarrow A[i]$
7. if $b > sum$
8. then $sum \leftarrow b$
9. $c \leftarrow i$ // 记录最大和的末项标号
10. return sum, c

时间复杂度 $O(n)$

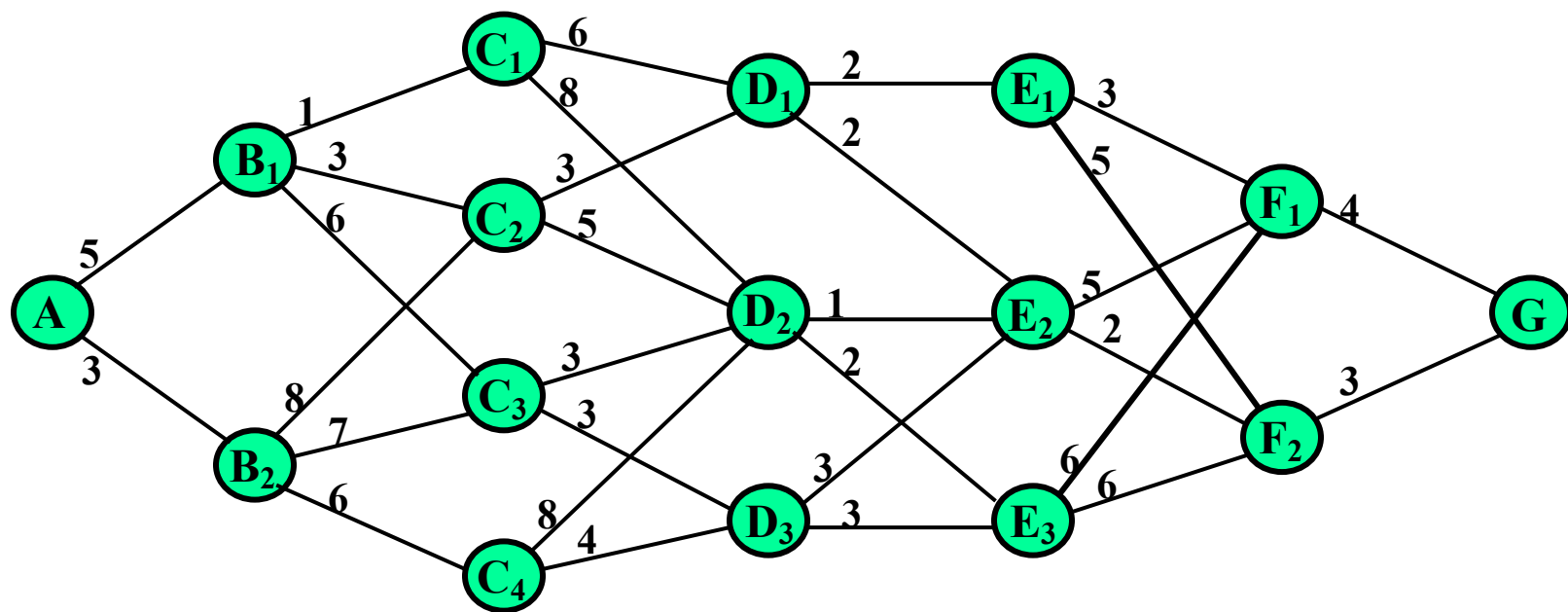
4.4.5 多段图问题

- 设 $G=(V, E)$ 是一个有向图，具有如下性质：图中的结点被划分成 k ($k \geq 2$) 个不相交的集合 V_i , $1 \leq i \leq k$, 其中 V_1 和 V_k 分别只有一个结点 s (源点) 和 t (汇点), 图中所有的边 $\langle u, v \rangle$ 均具有如下性质：若 $u \in V_i$, 则 $v \in V_{i+1}$, $1 \leq i \leq k$, 且每条边 $\langle u, v \rangle$ 均附有成本 $c(u, v)$ 。从 s 到 t 的一条路径成本是这条路径上边的成本和。多段图问题就是求 s 到 t 的最小成本路径。



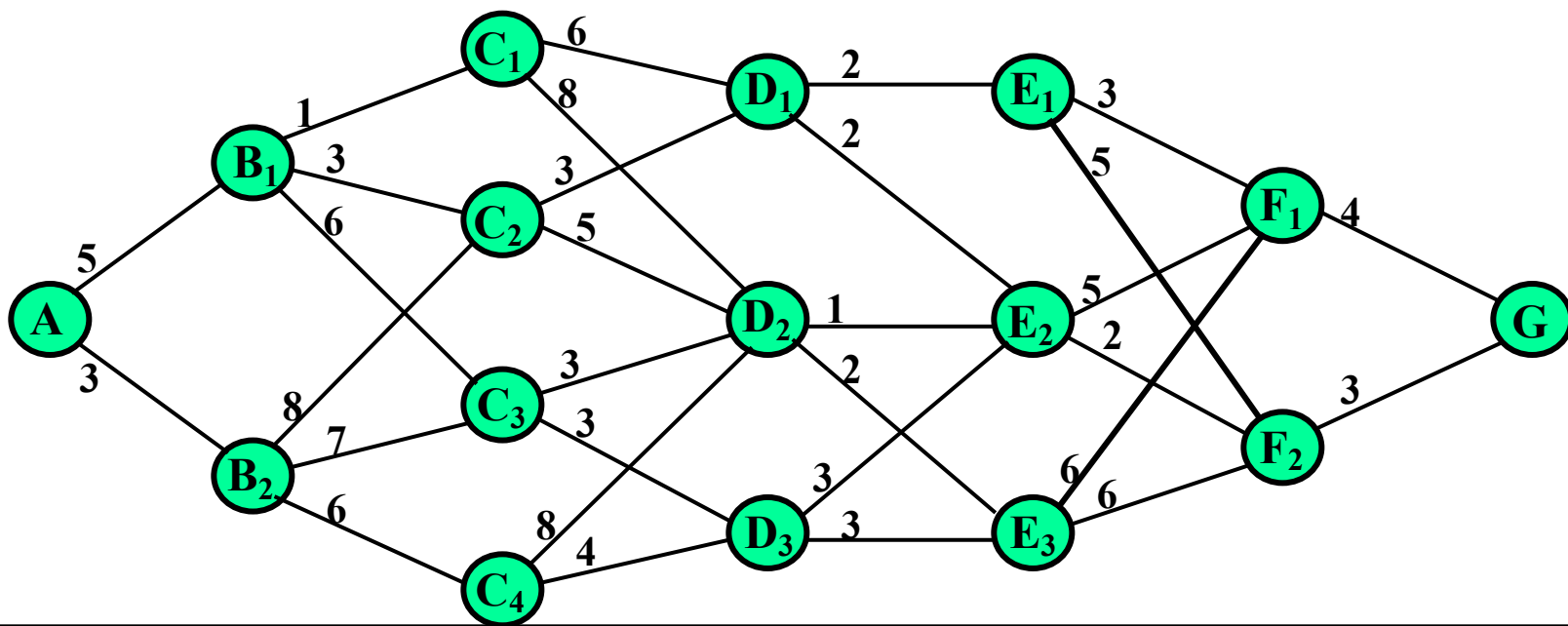
多段图问题的例子

- 从A点铺设一条煤气管道到G点，由于距离过长，必须经过五次加压才能保证终端用户的压力。经过前期调查确定每个加压站的五个备选位置，由于地理条件限制某些地点之间不适合建管道，可选地及其管线连接关系如下图：



多段图问题的例子

- 枚举算法：列出每条可能的路线，然后算出每条路的长度，从中选择最优路线。缺点是线路太多，随点数增加很快。





思路一

- 设 $P(i,j)$:一条从 V_i 中结点 j 到汇点 t 的最短路径;
- $COST(i,j)$: $P(i,j)$ 的成本
- $D(i,j)$: V_i 中结点 j (状态)下作的最优决策

$$COST(i,j) = \begin{cases} 0 & i = k \\ \min_{l \in V_{i+1}, \langle j,l \rangle \in E} \{c(j,l) + COST(i+1,l)\} & i < k \end{cases}$$





思路二

- $BP(i,j)$: 一条从源点 s 到 V_i 中结点 j 的最短路径;
- $BCOST(i,j)$: $BP(i,j)$ 的成本
- $D(i,j)$: V_i 中结点 j (状态) 下作的最优决策

$$BCOST(i,j) = \begin{cases} 0 & i = 1 \\ \min_{l \in V_{i-1}, \langle l,j \rangle \in E} \{BCOST(i-1,l) + c(l,j)\} & i > 1 \end{cases}$$





多段图的应用—资源分配问题

问题描述:

- 把 n 个资源分配给 r 个项目。如果把 j 个资源分配给项目 i , 所获得的净利记为 $N(i,j)$ 。要求按使得总净利达到最大值的方法把 n 个资源分配给这 r 个项目.



建模思路

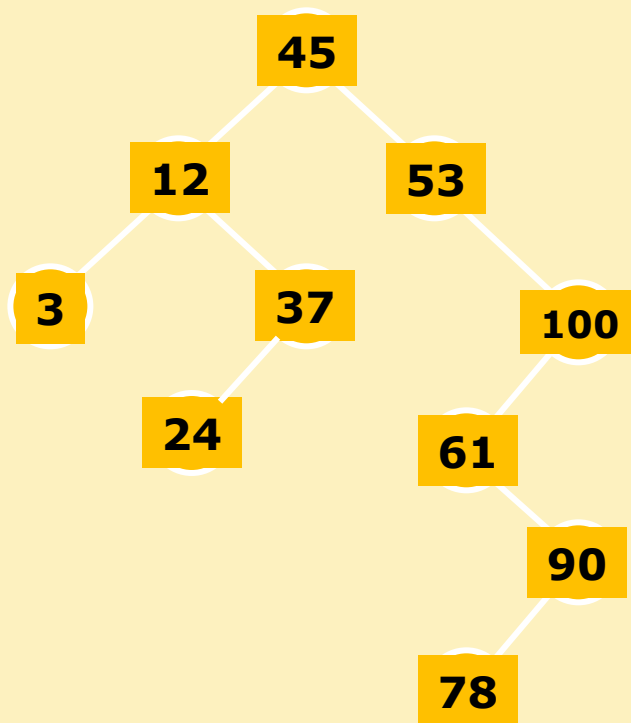
- 将该问题转化为 $r+1$ 段图问题。1到 r 之间的段 i 表示项目 i .
- 结点表示:
 - (1) $s=V(1,0)$, $t=V(r+1,n)$
 - (2) 段 i 为2到 r 时,每段有 $n+1$ 个结点 $V(i,j)$, $j=0,1,\dots,n$. $V(i,j)$ 表示共把 j 个资源分配给了项目 $1,2,\dots,i-1$
- 边的形式: $\langle V(i,j), V(i+1,l) \rangle$, $j \leq l$, $1 \leq i \leq r$
- 边的成本:
 - $j \leq l$, $1 \leq i < r$ 时, $\langle V(i,j), V(i+1,l) \rangle$ 的成本为: $N(i, l-j)$
 - $j \leq n$, $i=r$ 时, 即边 $\langle V(r,j), V(r+1,n) \rangle$ 的成本为: $\max \{N(r,p) \mid 0 \leq p \leq n-j\}$

资源最优分配方案由 s 到 t 的一条最大成本路径确定。改变所有边的成本符号, 即可用多段图问题的算法求解。



4.4.6 最优二分检索树问题

- 什么是二叉检索树?
 - 若它的左子树不空，则左子树上所有节点的值均小于它的根节点的值；
 - 若它的右子树不空，则右子树上所有节点的值均大于它的根节点的值；
 - 它的左、右子树也分别为二叉检索树
- 在随机的情况下，二叉查找树的平均查找长度和 $\log n$ 是等数量级的。

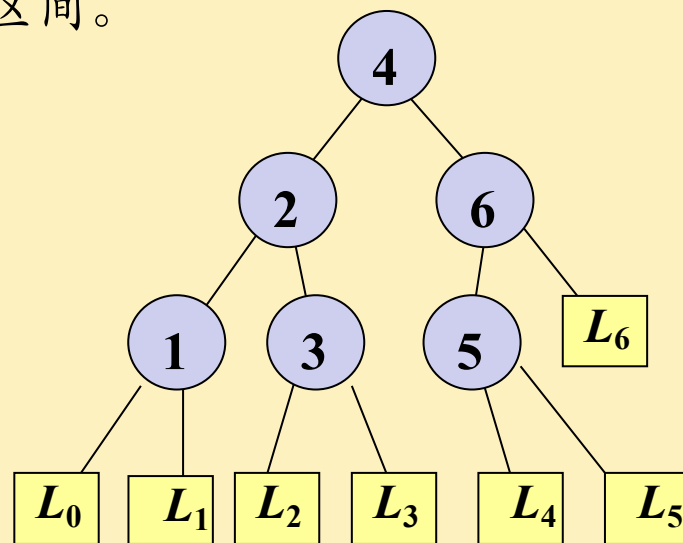


4.4.6 最优二分检索树问题

设集合 S 为排序的 n 个元素 $x_1 < x_2 < \dots < x_n$ ，将这些元素存储在一棵二叉树的结点上，以查找 x 是否在这些数中. 如果 x 不在，确定 x 在那个空隙。树的叶结点是形如 (x_i, x_{i+1}) 的开区间。

检索方法：

1. 初始， x 与根元素比较；
2. $x <$ 根元素，递归进入左子树；
3. $x >$ 根元素，递归进入右子树；
4. $x =$ 根元素，算法停止，输出 x ；
5. x 到达叶结点，算法停止，输出 x 不在数组中。



返回结果的两种情形：

- (1) 在二叉搜索树的内结点中找到 $x = x_i$ 。
- (2) 在二叉搜索树的叶结点中确定 $x \in (x_i, x_{i+1})$ 。

$S = \langle 1, 2, 3, 4, 5, 6 \rangle$

4.3.6 最优二分检索树问题

- 设第一种情形中找到元素 $x=x_i$ 的概率为 b_i ；在第二种情形中确定 $x \in (x_i, x_{i+1})$ 的概率为 a_i 。并约定 $x_0 = -\infty$, $x_{n+1} = +\infty$ 。显然有

$$a_i \geq 0, 0 \leq i \leq n; \quad b_j \geq 0, 0 \leq j \leq n; \quad \sum_{i=0}^n a_i + \sum_{j=1}^n b_j = 1$$

- $(a_0, b_1, a_1, \dots, b_n, a_n)$ 称为集合 S 的存取概率分布。
- 在表示 S 的二分检索树 T 中，设存储元素 x_i 的结点深度为 c_i ；存储叶结点 (x_j, x_{j+1}) 的结点深度为 d_j ，则

$$p = \sum_{i=1}^n b_i(1 + c_i) + \sum_{j=0}^n a_j d_j$$

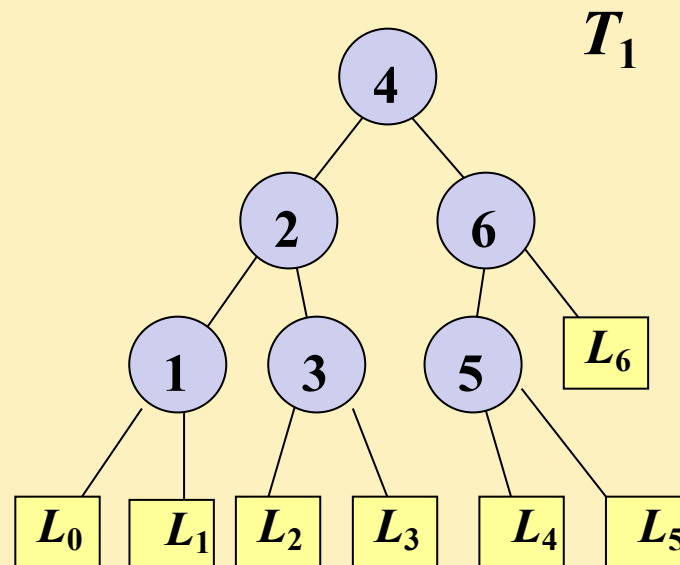
表示在二分检索树 T 中做一次搜索所需要的平均比较次数。 p 又称为二分检索树 T 的加权平均路长。在一般情况下，不同的二叉树的加权平均路长是不相同的。

实例

$S = \langle 1, 2, 3, 4, 5, 6 \rangle$

S 的存取概率分布 = $\langle 0.04, 0.1, 0.01, 0.2, 0.05, 0.2, 0.02, 0.1, 0.02, 0.1, 0.07, 0.05, 0.04 \rangle$

红色数组为内部节点存取概率；蓝色为叶子节点存取概率。



$$\begin{aligned} p(T_1) &= [1 * 0.1 + 2 * (0.2 + 0.05) + 3 * (0.1 + 0.2 + 0.1)] \text{ -- 内部节点加权路径长度} \\ &\quad + [3 * (0.04 + 0.01 + 0.05 + 0.02 + 0.02 + 0.07) + 2 * 0.04] \text{ -- 外部节点加权路径长度} \\ &= 1.8 + 0.71 = 2.51 \end{aligned}$$

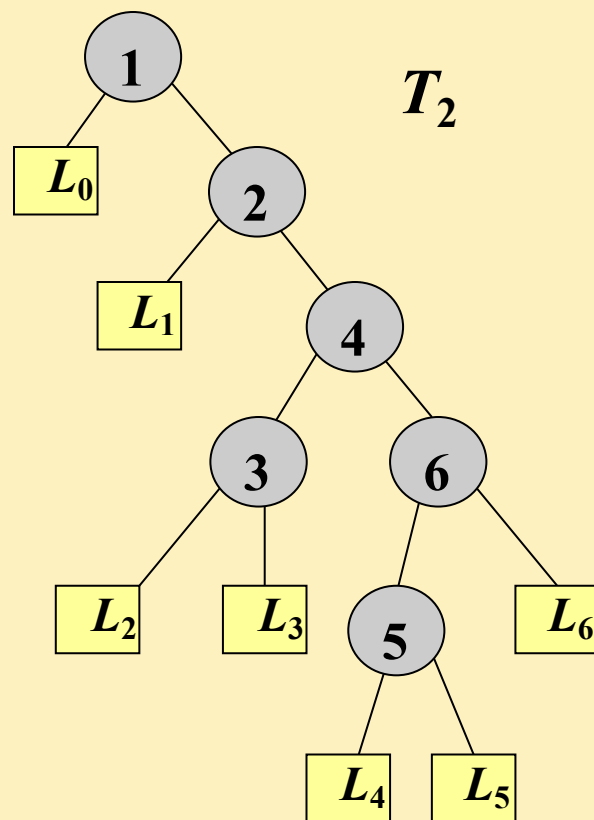
实例

$S = \langle 1, 2, 3, 4, 5, 6 \rangle$

S 的存取概率分布 = $\langle 0.04, 0.1, 0.01, 0.2, 0.05, 0.2, 0.02, 0.1, 0.02, 0.1, 0.07, 0.05, 0.04 \rangle$

红色数组为内部节点存取概率；蓝色为叶子节点存取概率。

$$\begin{aligned} p(T_2) &= [1*0.1 + 2*0.2 + 3*0.1 \\ &\quad + 4*(0.2 + 0.05) + 5*0.1] \\ &\quad + [1*0.04 + 2*0.01 \\ &\quad + 4*(0.05 + 0.02 + 0.04) \\ &\quad + 5*(0.02 + 0.07)] \\ &= 2.3 + 0.95 = 3.25 \end{aligned}$$



4.3.6 最优二分检索树问题

最优二分检索树问题

- 对于有序集 S 及其存取概率分布 $(a_0, b_1, a_1, \dots, b_n, a_n)$, 在所有表示有序集 S 的二分检索树中找出一棵具有最小平均加权路长的二分检索树。
- 有 n 个节点的二叉树的个数为: $\Omega(4^n/n^{3/2})$ 。因此, 穷举搜索法的时间复杂度为指数级。

4.3.6 最优二分检索树问题

数据集 $S = \langle x_1, x_2, \dots, x_n \rangle$, $x_1 < x_2 < \dots < x_n$

存取概率分布 $P = \langle a_0, b_1, a_1, b_2, \dots, a_n, b_{n+1} \rangle$

结点 x_i 在 T 中的深度是 $d(x_i)$, $i=1, 2, \dots, n$,

空隙 L_j 的深度为 $d(L_j)$, $j=0, 1, \dots, n$,

平均比较次数为:

$$t = \sum_{i=1}^n b_i (1 + d(x_i)) + \sum_{j=0}^n a_j d(L_j)$$

问题: 给定数据集 S 和相关存取概率分布 P , 求一棵最优的(即平均比较次数最少的) 二分检索树.

子问题划分

记: $S[i,j] = \langle x_i, x_{i+1}, \dots, x_j \rangle$ 是 S 以 i 和 j 作为边界的子数据集
 $P[i,j] = \langle a_{i-1}, b_i, a_i, b_{i+1}, \dots, b_j, a_j \rangle$ 是对应 $S[i,j]$ 存取概率分布

例: $S = \langle A, B, C, D, E \rangle$

$P = \langle 0.04, \underline{0.1}, 0.02, \underline{0.3}, 0.02, \underline{0.1}, 0.05, \underline{0.2}, 0.06, \underline{0.1}, 0.01 \rangle$

$S[2,4] = \langle B, C, D \rangle$

$P[2,4] = \langle 0.02, \underline{0.3}, 0.02, \underline{0.1}, 0.05, \underline{0.2}, 0.06 \rangle$

子问题划分: 以 x_k 作为根, 划分成两个子问题:

$S[i, k-1], P[i, k-1]$

$S[k+1, j], P[k+1, j]$

例: 以 B 为根, 划分成以下子问题:

$S[1,1] = \langle A \rangle, P[1,1] = \langle 0.04, \underline{0.1}, 0.02 \rangle$

$S[3,5] = \langle C, D, E \rangle, P[3,5] = \langle 0.02, \underline{0.1}, 0.05, \underline{0.2}, 0.06, \underline{0.1}, 0.01 \rangle$

最优子结构性质

- 设 T_{ij} 是有序集合 $S[i,j] = \langle x_i, x_{i+1}, \dots, x_j \rangle$ 关于存取概率分布 $P[i,j] = \langle a_{i-1}, b_i, a_i, b_{i+1}, \dots, b_j, a_j \rangle$ 的一棵最优二分检索树，设其平均加权路径长为 p_{ij} 。并设 T_{ij} 的根结点存储元素 x_m ，其左右子树 T_{left} 和 T_{right} 的平均加权路径长分别为 p_{left} 和 p_{right} 。
- 由于 T_{left} 和 T_{right} 中结点深度是他们在 T_{ij} 中的结点深度减1，故有：
$$P_{ij} = p_{\text{left}} + p_{\text{right}} + w_{ij} \quad \text{其中, } w_{ij} = a_{i-1} + b_i + \dots + b_j + a_j。$$
- 由于 T_{left} 是关于集合 $\{x_i, \dots, x_{m-1}\}$ 的一棵二分检索树，故 $p_{\text{left}} \geq p_{i(m-1)}$ 。若 $p_{\text{left}} > p_{i(m-1)}$ ，则用 $T_{i(m-1)}$ 替换 T_{left} 可得到平均路长比 T_{ij} 更小的二分检索树。这与 T_{ij} 是最优二分检索树矛盾。故 T_{left} 是一个最优二分检索树。同理可证 T_{right} 也是一棵最优二分检索树。
- 因此最优二分检索树问题具有最优子结构性质。

求最优值的递推方程

设 $m[i,j]$ 是相对于输入 $S[i,j]$ 和 $P[i,j]$ 的最优二分检索树的平均比较次数, 令

$$w[i, j] = \sum_{p=i-1}^j a_p + \sum_{q=i}^j b_q$$

是 $P[i,j]$ 中所有概率 (包括数据元素与空隙) 之和

则求最优值的递推方程:

$$m[i, j] = \min_{i \leq k \leq j} \{m[i, k-1] + m[k+1, j] + w[i, j]\}, \quad 1 \leq i \leq j \leq n$$

$$m[i, i-1] = 0, \quad i = 1, 2, \dots, n$$

证明

$m[i,j]_k$: 根为 x_k 时的二分检索树平均比较次数的最小值

$$\begin{aligned} & m[i,j]_k \\ &= (m[i,k-1] + w[i,k-1]) + (m[k+1,j] + w[k+1,j]) + 1 \times b_k \\ &= (m[i,k-1] + m[k+1,j]) + (w[i,k-1] + b_k + w[k+1,j]) \\ &= (m[i,k-1] + m[k+1,j]) + \left(\sum_{p=i-1}^{k-1} a_p + \sum_{q=i}^{k-1} b_q \right) + b_k + \left(\sum_{p=k}^j a_p + \sum_{q=k+1}^j b_q \right) \\ &= (m[i,k-1] + m[k+1,j]) + \sum_{p=i-1}^j a_p + \sum_{q=i}^j b_q \\ &= m[i,k-1] + m[k+1,j] + w[i,j] \end{aligned}$$

平均比较次数：在所有 k 的情况下 $m[i,j]_k$ 的最小值，

$$m[i,j] = \min \{ m[i,j]_k \mid i \leq k \leq j \}$$

例

$$m[i, j] = \min_{i \leq k \leq j} \{m[i, k-1] + m[k+1, j] + w[i, j]\} \quad 1 \leq i \leq j \leq n$$

$$m[i, i-1] = 0$$

$$m[1,1] = 0.16, \quad m[3,5] = 0.88$$

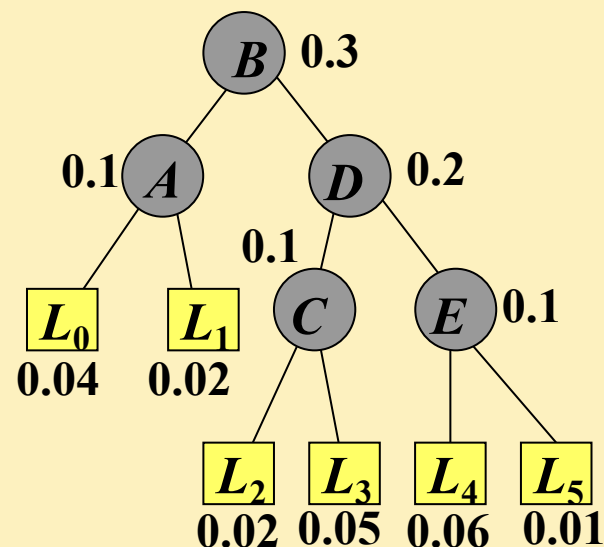
$$m[1,5] = 1 + \min \{m[1, k-1] + m[k+1, 5]\}$$

$$= 1 + \{m[1,1] + m[3,5]\}$$

$$= 1 + \{0.16 + 0.88\} = 2.04$$

算法复杂性估计:

$$T(n) = O(n^3) \quad S(n) = O(n^2)$$



4.3.7 投资问题

m 元钱, n 项投资, $f_i(x)$: 将 x 元投入第 i 项项目的效益

目标函数 $\max \{f_1(x_1) + f_2(x_2) + \dots + f_n(x_n)\}$

约束条件 $x_1 + x_2 + \dots + x_n = m, x_i \in \mathbf{N}$

实例: 5万元钱, 4个项目, 效益函数如下表所示

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

动态规划方程

设 $F_k(x)$ 表示 x 元钱投给前 k 个项目的最大效益

多步判断：假设知道 p 元钱 ($p \leq x$) 投给前 $k-1$ 个项目的最大效益，决定 x 元钱投给前 k 个项目的分配方案。

递推方程和边界条件

$$F_k(x) = \max_{0 \leq x_k \leq x} \{f_k(x_k) + F_{k-1}(x - x_k)\} \quad k > 1$$

$$F_1(x) = f_1(x)$$

动态规划方程

x	$F_1(x) \ x_1(x)$	$F_2(x) \ x_2(x)$	$F_3(x) \ x_3(x)$	$F_4(x) \ x_4(x)$
1	11 1	11 0	11 0	20 1
2	12 2	12 0	13 1	31 1
3	13 3	16 2	30 3	33 1
4	14 4	21 3	41 3	50 1
5	15 5	26 4	43 4	61 1

解 $x_1=1, x_2=0, x_3=3, x_4=1 \ F_4(5)=61$

算法的复杂度分析

表中有 m 行 n 列, 共计 mn 项

$$F_k(x) = \max_{0 \leq x_k \leq x} \{f_k(x_k) + F_{k-1}(x - x_k)\} \quad k > 1$$

$$F_1(x) = f_1(x)$$

对第 $F_k(x)$ 项 ($2 \leq k \leq n, 1 \leq x \leq m$) 需要 $x+1$ 次加法, x 次比较

加法次数
$$\sum_{k=2}^n \sum_{x=1}^m (x+1) = \frac{1}{2}(n-1)m(m+3)$$

比较次数
$$\sum_{k=2}^n \sum_{x=1}^m x = \frac{1}{2}(n-1)m(m+1)$$

$$W(n) = O(nm^2)$$